

Lab 5

Introduction to Databases



Relationships

Relationships define how tables in a relational database are connected and interact through foreign keys, ensuring data integrity and enabling efficient data retrieval by allowing data to be linked across multiple tables.

Type of Relationships : There are different types of relationships: one-to-one, one-to-many, many-to-many.

1. One-to-One Relationship

Each record in Table A is associated with one and only one record in Table B, and vice versa.

Setup: Include a foreign key in one of the tables that references the primary key of the other table.

For example: Tables `users` and `user_profiles`, where each user has a single corresponding profile.

[illegible]

```
CREATE TABLE users (  
    user_id NUMBER PRIMARY KEY,  
    username VARCHAR2(50));
```

```
CREATE TABLE user_profiles (  
    profile_id NUMBER PRIMARY KEY,  
    user_id NUMBER UNIQUE,  
    profile_data VARCHAR2(255),  
    FOREIGN KEY (user_id) REFERENCES users(user_id));
```

2. One-to-Many Relationship

Each record in Table A can be associated with multiple records in Table B, but each record in Table B is associated with only one record in Table A.

- Setup:** Include a foreign key in the "many" side table (Table B) that references the primary key of the "one" side table (Table A).
- For example:** Tables departments and employees, where each department can have multiple employees, but each employee belongs to one department.

Departments			Employees		
department_id	department_name		employee_id	employee_name	department_id
d1	technical		e01	Ramesh	d3
d2	accounts		e02	Riya	d1
d3	pr		e03	Neha	d2
d4	product management		e04	Mayank	d1
			e05	Kritila	d4
			e06	Anuj	d4
			e07	Sam	d1
			e08	Gurpreet	d2
			foreign key		
			primary key		

```
CREATE TABLE departments (  
    department_id NUMBER PRIMARY KEY,  
    department_name VARCHAR2(50));  
  
CREATE TABLE employees (  
    employee_id NUMBER PRIMARY KEY,  
    employee_name VARCHAR2(50),  
    department_id NUMBER ,  
    FOREIGN KEY (department_id) REFERENCES departments(department_id));
```

3. Many-to-Many Relationship

Each record in Table A can be associated with multiple records in Table B, and vice versa.

Setup: Create an intermediate table (also known as a junction or linking table) that contains foreign keys referencing both related tables.

For example: Tables students and courses, where each student can enroll in multiple courses, and each course can have multiple students.

[illegible]

```
CREATE TABLE students (  
    student_id NUMBER PRIMARY KEY,  
    student_name VARCHAR2(50));
```

```
CREATE TABLE courses (  
    course_id NUMBER PRIMARY KEY,  
    course_name VARCHAR2(50));
```

```
CREATE TABLE student_courses (  
    student_id NUMBER,  
    course_id NUMBER,  
    PRIMARY KEY (student_id, course_id),  
    FOREIGN KEY (student_id) REFERENCES students(student_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id));
```


Join tables and Retrieving data from more than one table

1. Create a two Tables:

Create a table Supplier with the following columns:

- supplier_id → NUMBER (Primary Key)
- supplier_name → VARCHAR2(50) Not Null

Create a table Product with the following columns:

- product_id → NUMBER
- product_name → VARCHAR2(50) Not Null
- supplier_id → NUMBER
- Price → NUMBER

2. Insert Data into a Table:

Insert the following Supplier into the table:

(100, 'IBM')
(200, 'HP')
(300, 'Microsoft')
(400, 'Apple')

Insert the following Product into the table:

(1, 'IPAD2', 400, 2400)
(2, 'IPHONE 4s', 400, 2500)
(3, 'MS Office 2012', 300, 1600)
(4, 'Color Printer', 100, 1500)

3. Retrieving the supplier_name , product_name, Price using **Join -- On**
4. Retrieving the supplier_name , product_name, Price using **Where**
5. Retrieving the supplier_name , product_name, Price **Where price more than 1000**
6. Retrieving the supplier_name , Price **Where product name is Apple**